

UNIVERSIDADE DE VIGO
Escola Superior de Enxeñaría Informática

Plataforma SaaS para la Gestión de Comunidades Energéticas

Asignatura Aplicaciones Web (ALS)
Autor Roberto Rodríguez Núñez — 53975479E
Profesor Baltasar García Pérez-Schofield
Curso 2025/2026

Índice

1. Visión General	1
2. Stack Tecnológico	1
3. Arquitectura	1
3.1. Patrón Application Factory	1
3.2. Blueprints Modulares	2
3.3. OIDs URL-safe	2
4. Modelo de Datos	2
4.1. Entidades	2
4.2. Relaciones	3
4.3. Almacenamiento en Sirope	3
4.4. Diagrama de Clases	4
5. Roles y Permisos	4
5.1. Roles Globales	4
5.2. Roles en Vivienda	4
5.3. Tabla de Permisos	5
5.4. Implementación	5
6. Integridad Referencial	5
7. Análisis Funcional	5
7.1. Actores	6
7.2. Casos de Uso	6
8. Rutas y Endpoints	7
9. Interfaz de Usuario y Diseño Visual	8
9.1. Plantillas Jinja2	8
9.2. Design System	8
9.3. Gráficas Chart.js	8
10.AJAX y Progressive Enhancement	9
10.1. Detección y CSRF en Servidor	9
10.2. Acciones AJAX Implementadas	9
11.Sistema de Notificaciones	9
11.1. Tipos de Notificación	9
11.2. Notificaciones Automáticas	10
11.3. Envío Manual	10
12.Testing	10
12.1. Configuración	10
12.2. Cobertura	10
12.3. Tipos de Test	10
13.Despliegue	11
13.1. Docker	11
13.2. Variables de Entorno	11

13.3. Primer Uso	11
14. Estructura del Proyecto	11
15. Criterios del Enunciado — Tabla de Cumplimiento	12

1 Visión General

La **plataforma** es una aplicación web SaaS de gestión de comunidades energéticas. Una comunidad energética es una agrupación de viviendas que comparten una batería de almacenamiento y, opcionalmente, instalaciones fotovoltaicas individuales. La plataforma permite a un *Superadmin* gestionar comunidades, viviendas, usuarios, baterías, cierres mensuales e incidencias; los *Usuarios normales* pueden consultar sus viviendas, ver sus cierres mensuales, crear incidencias y gestionar sus notificaciones.

El sistema actúa como capa de gestión administrativa: centraliza los datos de entrada de los algoritmos de optimización energética (viviendas, coeficientes de reparto, parámetros de batería) y presenta los resultados de simulación (cierres mensuales) a través de una interfaz web multi-rol.

2 Stack Tecnológico

Tabla 1: Tecnologías empleadas en la plataforma.

Tecnología	Versión	Rol
Python	3.11+	Lenguaje principal
Flask	3.0.3	Framework web (patrón Application Factory)
Jinja2	con Flask	Motor de plantillas HTML
Sirope	0.3.1	ORM sobre Redis (serialización de objetos Python)
Redis	7	Base de datos (almacén clave-valor)
Flask-Login	0.6.3	Gestión de sesiones y autenticación
Flask-WTF	1.2.1	Protección CSRF y formularios
WTForms	3.1.2	Validación de formularios del lado servidor
Werkzeug	≥3.0	Hashing de contraseñas (PBKDF2)
Pico CSS	2 (CDN)	Framework CSS base
Chart.js	4 (CDN)	Gráficas interactivas (línea, barras, doughnut)
Docker	—	Contenerización (Dockerfile + docker-compose)
pytest	—	Suite de tests automatizados

3 Arquitectura

3.1 Patrón Application Factory

La aplicación se construye con el patrón *Application Factory* en `app/__init__.py`. La función `create_app(config_name)` realiza, en orden: (1) crea la instancia Flask y carga la configuración según el entorno; (2) conecta con Redis e instancia Sirope; (3) inicializa Flask-Login y CSRFProtect; (4) registra globales Jinja2 (`oid_to_safe`, `csrf_token_hidden`); (5) configura el `user_loader`; (6) añade un `context_processor` que inyecta en cada plantilla el contador de notificaciones no leídas y la lista de comunidades del usuario para el navbar; (7) registra los 10 blueprints; y (8) configura manejadores de error personalizados (401, 403, 404, 500).

Este patrón permite crear múltiples instancias con configuración diferente (tests vs. producción), evita estado global y facilita la inyección de dependencias.

3.2 Blueprints Modulares

Se aplica el criterio de *un blueprint por entidad o grupo funcional*:

Tabla 2: Blueprints registrados en la aplicación.

Blueprint	Prefijo URL	Responsabilidad
auth	/	Login, logout, registro, perfil, cambio de contraseña
main	/	Dashboard según rol
comunidades	/comunidades	CRUD de comunidades
viviendas	/viviendas	CRUD de viviendas
usuarios	/usuarios	Listado, detalle, asignación y borrado
accesos	/accesos	CRUD de accesos (relación usuario-vivienda)
baterias	/baterias	CRUD de baterías y cambio de estado
cierres	/cierres	CRUD de cierres mensuales
notificaciones	/notificaciones	Listado, lectura, eliminación y envío
incidencias	/incidencias	CRUD de incidencias y respuestas

Cada blueprint tiene su propio directorio con `__init__.py`, `routes.py` y, cuando procede, `forms.py`.

3.3 OIDs URL-safe

Sirope genera OIDs con formato `namespace.Clase@número` (ej: `app.models.comunidad.Comunidad@3`). Los puntos y la arroba no son válidos en rutas Flask, por lo que se implementa una conversión bidireccional en `app/helpers.py`:

Listing 1: Conversión de OIDs a formato URL-safe.

```

1 def oid_to_safe(oid) -> str:
2     return str(oid).replace('.', '-dot-').replace('@', '-at-')
3
4 def oid_from_safe(safe: str):
5     from sirope import OID
6     return OID.from_text(safe.replace('-dot-', '.').replace('-at-', '@'))

```

`oid_to_safe` se registra como global Jinja2, siendo usable directamente en plantillas: `{{ oid_to_safe(obj.__oid__) }}`.

4 Modelo de Datos

4.1 Entidades

El modelo consta de **7 entidades** (8 clases Python, incluyendo AccesoVivienda como relación asociativa):

Tabla 3: Entidades del modelo de datos.

Entidad	Atributos principales
Usuario	nombre, email, password_hash, rol_global, fecha_registro
Comunidad	nombre, ubicacion, fecha_constitucion, estado, descripcion
Vivienda	comunidad_oid, identificador, cups, potencia_contratada_kw, coeficiente_reparto, tiene_paneles, potencia_pico_paneles_kwp
AccesoVivienda	usuario_oid, vivienda_oid, rol_en_vivienda, fecha_incorporacion
Bateria	comunidad_oid, capacidad_nominal_kwh, capacidad_util_actual_kwh, ciclos_acumulados, fabricante, modelo, estado
CierreMensual	vivienda_oid, mes, consumo_total_kwh, autoconsumo_directo_kwh, energia_de_bateria_kwh, ahorro_eur, factura_escenario_base/real_eur
Incidencia	vivienda_oid, comunidad_oid, usuario_oid, titulo, tipo, estado, respuesta_admin
Notificacion	usuario_oid, tipo, titulo, mensaje, entidad_relacionada_oid, leida

4.2 Relaciones

Tabla 4: Relaciones entre entidades.

Relación	Cardinalidad	Descripción
Comunidad → Vivienda	1:N	Una comunidad agrupa múltiples viviendas
Comunidad → Bateria	1:0..1	Una comunidad tiene como máximo una batería
Comunidad → Incidencia	1:N	Las incidencias se vinculan a una comunidad
Usuario ↔ Vivienda	N:M	Relación mediada por AccesoVivienda
Vivienda → CierreMensual	1:N	Historial de cierres por vivienda
Vivienda → Incidencia	1:N	Incidencias asociadas a una vivienda
Usuario → Incidencia	1:N	El usuario abre incidencias
Usuario → Notificacion	1:N	Notificaciones dirigidas a un usuario

La relación **Usuario** ↔ **Vivienda** es N:M y se representa mediante la clase **AccesoVivienda**, que almacena también el rol dentro de la vivienda (**titular**, **convivente**, **solo_lectura**).

4.3 Almacenamiento en Sirope

Sirope serializa objetos Python como JSON en Redis. No existe esquema fijo ni migraciones: las relaciones se mantienen como strings OID en los atributos de cada objeto (p. ej. `vivienda.comunidad_oid = str(comunidad.__oid__)`). La integridad referencial es responsabilidad del código de aplicación (véase sección 6).

4.4 Diagrama de Clases

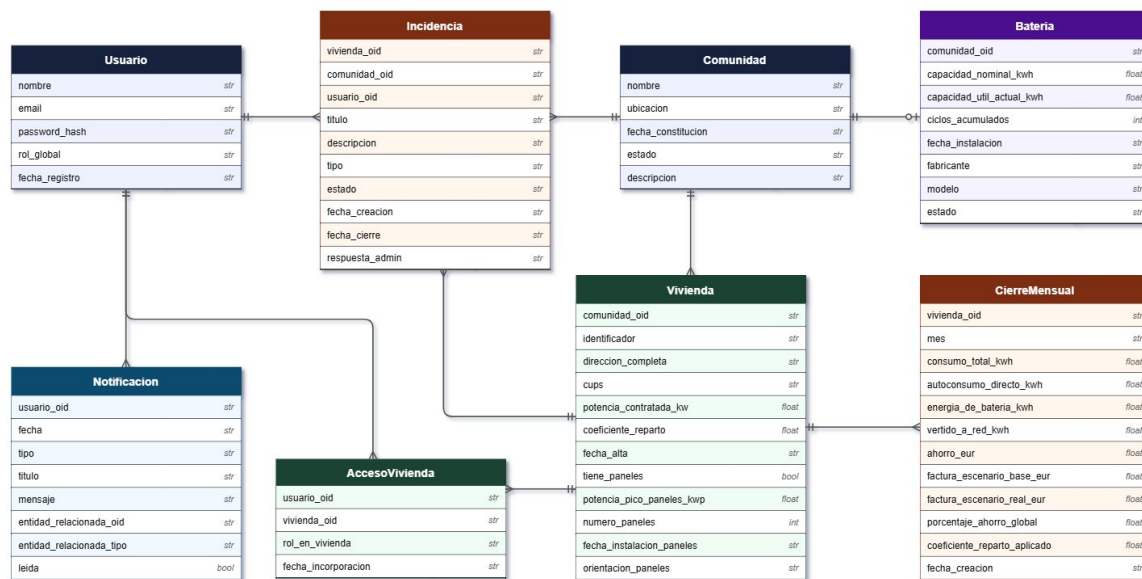


Figura 1: Diagrama de clases UML: 7 entidades con atributos, AccesoVivienda como relación asociativa N:M, y 8 relaciones con cardinalidad.

5 Roles y Permisos

5.1 Roles Globales

Tabla 5: Roles globales del sistema.

Rol	Descripción
superadmin	Administrador total. El primer usuario registrado obtiene este rol automáticamente.
normal	Usuario estándar con acceso limitado a sus viviendas y comunidades.

5.2 Roles en Vivienda

Dentro de cada vivienda, el rol se almacena en `AccesoVivienda.rol_en_vivienda`: **titular** (propietario; no eliminable si es el único), **convivente** (acceso lectura/escritura) y **solo_lectura** (consulta únicamente).

5.3 Tabla de Permisos

Tabla 6: Permisos por operación y rol.

Operación	Superadmin	Usuario normal
CRUD comunidades	✓	×
Ver comunidades	Todas	Solo las tuyas
CRUD viviendas	✓	×
CRUD accesos	✓	×
CRUD batería	✓	×
CRUD cierres mensuales	✓	×
Ver cierres	Todas	Solo sus viviendas
Crear incidencia	✓	En sus viviendas
Gestionar incidencias	✓	×
Enviar notificaciones	✓	×
Ver/eliminar notificaciones	✓	Solo las tuyas
Gestionar usuarios	✓	×

5.4 Implementación

Los permisos se aplican mediante decoradores en `app/decorators.py`:

- `@superadmin_required` — aborta 401 si no autenticado, 403 si no es superadmin.
- `@acceso_vivienda_required(param)` — aborta 403 si el usuario no tiene `AccesoVivienda` para esa vivienda (el superadmin tiene acceso implícito).

6 Integridad Referencial

Al no disponer de claves foráneas nativas en Sirope/Redis, la integridad se implementa manualmente en `app/helpers.py` mediante tres mecanismos:

Borrados en cascada. Al eliminar una **Comunidad**, se borran en cascada sus viviendas, accesos, cierres e incidencias asociadas, la batería y las notificaciones relacionadas. Al eliminar una **Vivienda**, se eliminan sus accesos, cierres e incidencias. Al eliminar un **Usuario** se eliminan sus accesos, pero se bloquea si es el único titular de alguna vivienda.

Guardia de último titular. La función `puede_borrar_usuario()` devuelve (`False`, `motivo`) si el usuario es el único titular de alguna vivienda, evitando dejar viviendas huérfanas.

Recálculo automático de coeficientes. Al añadir, editar o eliminar una vivienda, la función `recalcular_coeficientes()` redistribuye los coeficientes de reparto entre todas las viviendas de la comunidad según:

$$\beta_i = \frac{P_{\text{contratada},i}}{\sum_j P_{\text{contratada},j}}$$

Si algún coeficiente cambia, se genera automáticamente una notificación a todos los usuarios con acceso a esa comunidad.

7 Análisis Funcional

7.1 Actores

El sistema contempla dos actores principales: el **Superadmin** (administrador técnico de la plataforma) y el **Usuario** (vecino de la comunidad sin perfil técnico).

7.2 Casos de Uso

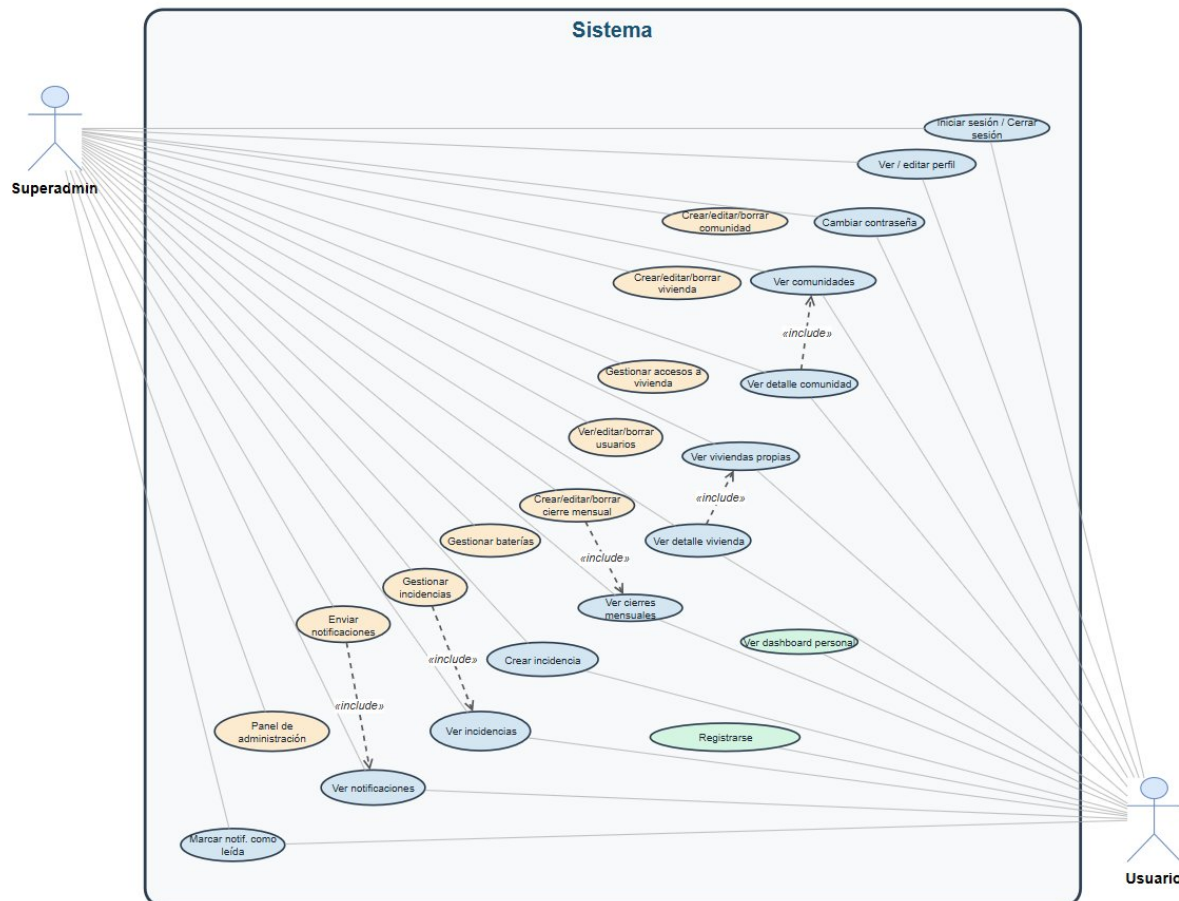


Figura 2: Diagrama de casos de uso: 2 actores, 23 casos de uso organizados por color (compartidos en azul, exclusivos Superadmin en naranja, exclusivos Usuario en verde) y relaciones «include».

Los casos de uso **compartidos** (accesibles por ambos actores) incluyen: iniciar/cerrar sesión, ver/editar perfil, cambiar contraseña, ver comunidades y viviendas propias, ver cierres mensuales, crear y ver incidencias, y gestionar notificaciones. Los **exclusivos del Superadmin** comprenden el CRUD completo de todas las entidades, la gestión de incidencias (respuesta y cambio de estado), el envío de notificaciones y el panel de administración. Los **exclusivos del Usuario** son el registro en la plataforma y el acceso al dashboard personal.

Se definen **5 relaciones «include»**:

- *Ver detalle comunidad* → *Ver comunidades*
- *Ver detalle vivienda* → *Ver viviendas propias*
- *Gestionar incidencias* → *Ver incidencias*
- *Crear/editar/borrar cierre mensual* → *Ver cierres mensuales*
- *Enviar notificaciones* → *Ver notificaciones*

8 Rutas y Endpoints

La aplicación expone **40 endpoints** distribuidos en 10 blueprints. Se presentan agrupados; la columna *Ruta* omite el prefijo del blueprint cuando es evidente.

Tabla 7: Endpoints: `auth_bp` y `main_bp`.

Método	Ruta	Descripción
GET, POST	/login	Inicio de sesión
GET	/logout	Cierre de sesión
GET, POST	/registro	Registro de usuario
GET, POST	/perfil	Edición de perfil
GET, POST	/cambiar-password	Cambio de contraseña
GET	/	Dashboard según rol (<code>main_bp</code>)

Tabla 8: Endpoints: `comunidades_bp` y `viviendas_bp`.

Método	Ruta	Descripción
<i>comunidades_bp</i> — prefijo <code>/comunidades</code>		
GET	/	Listado con buscador
GET, POST	/nueva	Crear comunidad
GET	/ <code><oid></code>	Detalle con viviendas
GET, POST	/ <code><oid></code> /editar	Editar comunidad
POST	/ <code><oid></code> /eliminar	Borrado en cascada
<i>viviendas_bp</i> — prefijo <code>/viviendas</code>		
GET	/	Listado global
GET, POST	/comunidad/ <code><oid></code> /nueva	Crear vivienda
GET	/ <code><oid></code>	Detalle con gráficas
GET, POST	/ <code><oid></code> /editar	Editar vivienda
POST	/ <code><oid></code> /eliminar	Borrado en cascada

Tabla 9: Endpoints: `usuarios_bp`, `accesos_bp` y `baterias_bp`.

Método	Ruta	Descripción
<i>usuarios_bp</i> — prefijo <code>/usuarios</code>		
GET	/	Listado con buscador
GET	/ <code><oid></code>	Detalle con accesos
POST	/ <code><oid></code> /asignar-vivienda	Asignar vivienda
POST	/ <code><oid></code> /eliminar	Borrar usuario
<i>accesos_bp</i> — prefijo <code>/accesos</code>		
GET	/vivienda/ <code><oid></code> /	Accesos de la vivienda
GET, POST	/vivienda/ <code><oid></code> /nuevo	Crear acceso
GET, POST	/ <code><oid></code> /editar	Editar rol
POST	/ <code><oid></code> /eliminar	Borrar (guardia titular)
<i>baterias_bp</i> — prefijo <code>/baterias</code>		
GET	/comunidad/ <code><oid></code> /	Detalle de batería
GET, POST	/comunidad/ <code><oid></code> /nueva	Crear batería
GET, POST	/ <code><oid></code> /editar	Editar batería
POST	/ <code><oid></code> /cambiar-estado	Cambiar estado (AJAX)
POST	/ <code><oid></code> /eliminar	Borrar batería

Tabla 10: Endpoints: `cierres_bp`, `incidencias_bp` y `notificaciones_bp`.

Método	Ruta	Descripción
<i>cierres_bp</i> — prefijo <i>/cierres</i>		
GET	<code>/vivienda/<oid>/</code>	Listado de cierres
GET, POST	<code>/vivienda/<oid>/nuevo</code>	Crear cierre mensual
GET	<code>/<oid></code>	Detalle con gráficas
POST	<code>/<oid>/eliminar</code>	Borrar cierre
<i>incidencias_bp</i> — prefijo <i>/incidencias</i>		
GET	<code>/</code>	Listado de incidencias
GET, POST	<code>/nueva</code>	Crear incidencia
GET	<code>/<oid></code>	Detalle
POST	<code>/<oid>/responder</code>	Responder (superadmin)
POST	<code>/<oid>/eliminar</code>	Borrar incidencia
<i>notificaciones_bp</i> — prefijo <i>/notificaciones</i>		
GET	<code>/</code>	Listado
POST	<code>/<oid>/leer</code>	Marcar leída (AJAX)
POST	<code>/leer-todas</code>	Marcar todas leídas (AJAX)
POST	<code>/<oid>/eliminar</code>	Borrar notificación
GET, POST	<code>/nueva</code>	Enviar (superadmin)

9 Interfaz de Usuario y Diseño Visual

9.1 Plantillas Jinja2

La aplicación cuenta con **34 plantillas Jinja2** organizadas por módulo. Todas extienden `base.html`, que incluye navbar, flash messages y bloque de contenido. La herencia de plantillas evita duplicación y garantiza coherencia visual.

Elementos comunes en todas las vistas:

- **Navbar sticky** con efecto glassmorphism (`backdrop-filter: blur(12px)`), logo, enlaces según rol y badge de notificaciones no leídas.
- **Buscador** en todos los listados, con filtrado realizado en servidor.
- **Flash messages** con borde izquierdo coloreado: verde (success), rojo (danger), ámbar (warning).
- **Páginas de error personalizadas** para los códigos 401, 403, 404 y 500.
- **Toggle switch** animado para el campo “¿Tiene paneles solares?” en el formulario de viviendas.

9.2 Design System

- **Pico CSS 2** como framework base, con tema claro personalizado.
- **Paleta Trade Republic Light**: fondo `#f7f7f7`, superficies `#ffffff`, texto `#0a0a0a`, acento verde `#18a070`, rojo `#d93025`.
- Variables CSS en `app/static/css/custom.css` para colores, sombras, radios y transiciones.

9.3 Gráficas Chart.js

En las vistas de detalle de vivienda y comunidad se renderizan cinco tipos de gráficas mediante `charts.js`:

- `line-ahorro` — evolución del ahorro mensual (área rellena).
- `bar-compare` — comparación factura base vs. factura real (barras agrupadas).
- `doughnut-mix` — distribución energía solar/batería/red.
- `bar-community` — ahorro por vivienda en la comunidad (barras horizontales).
- `line-ahorro-com` — evolución del ahorro comunitario mensual.

10 AJAX y Progressive Enhancement

Todos los formularios de borrado y acciones rápidas funcionan en modo dual:

- **Sin JavaScript:** POST estándar; el servidor responde con `flash()` + `redirect()`.
- **Con JavaScript:** `ajax.js` intercepta formularios con `data-ajax-action`, envía `fetch()` con cabecera `X-Requested-With: XMLHttpRequest` y el servidor devuelve JSON. El DOM se actualiza sin recarga.

10.1 Detección y CSRF en Servidor

Listing 2: Detección de petición AJAX y configuración CSRF.

```
1 def is_xhr(req=None) -> bool:
2     return req.headers.get('X-Requested-With') == 'XMLHttpRequest'
3
4 # settings.py
5 WTF_CSRF_HEADERS = ['X-CSRFToken']
```

El token CSRF se lee del meta tag `<meta name=csrf-token>` y se incluye automáticamente en el `FormData` de cada petición AJAX, garantizando protección incluso en operaciones sin recarga de página.

10.2 Acciones AJAX Implementadas

- Borrado de cualquier entidad (elimina el elemento del DOM con atributo `data-removable`).
- Marcar notificación como leída (actualiza la tarjeta y decrementa el badge del navbar: `data-ajax-notif="decrement"`).
- Marcar todas las notificaciones como leídas (`data-ajax-mark-all`).
- Cambiar estado de batería (reflejo inmediato en la UI).

11 Sistema de Notificaciones

11.1 Tipos de Notificación

Tabla 11: Tipos de notificación y cuándo se generan.

Tipo	Cuándo se genera
<code>cierre_disponible</code>	Al crear un cierre mensual para una vivienda
<code>cambio_coeficiente</code>	Al recalcular coeficientes si el valor cambia
<code>bateria_mantenimiento</code>	Al cambiar el estado de una batería
<code>incidencia</code>	Al crear o responder una incidencia
<code>general</code>	Enviada manualmente por el superadmin

11.2 Notificaciones Automáticas

La función `notificar_a_comunidad()` envía una notificación a todos los usuarios con acceso a alguna vivienda de la comunidad afectada. En el caso de cambios de coeficiente, solo se genera notificación si el valor realmente varía, evitando spam.

11.3 Envío Manual

El superadmin puede enviar notificaciones de tipo **general** a un usuario específico (seleccionable en dropdown) o a todos los usuarios del sistema (opción `__todos__`). El formulario valida título (2–120 caracteres) y mensaje (2–1000 caracteres).

Un `context_processor` inyecta el contador de notificaciones no leídas en todos los templates, mostrando un badge numérico en el navbar que se actualiza vía AJAX al marcar notificaciones.

12 Testing

12.1 Configuración

Los tests usan pytest con una configuración dedicada que crea una instancia de Redis limpia (`REDIS_DB=15`) para cada ejecución. Las *fixtures* en `confest.py` proporcionan instancias listas de todas las entidades: `app`, `client`, `superadmin`, `usuario`, `comunidad`, `vivienda`, `acceso`, `bateria`, `cierre`, `incidencia`, `notificacion`.

12.2 Cobertura

170 tests organizados en **43 clases** y **11 ficheros**:

Tabla 12: Ficheros de test y cobertura.

Fichero	Clases	Tests	Qué cubre
<code>test_auth.py</code>	4	22	Login, registro, perfil, contraseña
<code>test_comunidades.py</code>	5	18	CRUD comunidades, listado, detalle
<code>test_viviendas.py</code>	5	21	CRUD viviendas, listado, detalle
<code>test_baterias.py</code>	1	10	CRUD batería, cambio de estado
<code>test_cierres.py</code>	4	16	CRUD cierres, duplicado de mes
<code>test_accesos.py</code>	3	6	Listado, creación y borrado
<code>test_usuarios.py</code>	4	13	Listado, detalle, borrado
<code>test_incidencias.py</code>	5	17	CRUD incidencias, respuestas
<code>test_notificaciones.py</code>	5	15	Lectura, borrado, envío, AJAX
<code>test_helpers.py</code>	5	12	OIDs, coeficientes, cascadas
<code>test_main.py</code>	2	6	Dashboard, páginas de error
Total	43	170	—

12.3 Tipos de Test

- **Tests de ruta:** verifican códigos HTTP (200, 302, 403) y contenido de respuesta.
- **Tests de permisos:** comprueban que usuarios sin autorización reciben 403.

- **Tests AJAX:** envían cabecera `X-Requested-With: XMLHttpRequest` y verifican la respuesta JSON.
- **Tests de integridad:** verifican que los borrados en cascada eliminan correctamente los objetos dependientes.
- **Tests de lógica de negocio:** recálculo de coeficientes y guardia de último titular.

13 Despliegue

13.1 Docker

El proyecto incluye `Dockerfile` y `docker-compose.yml` para despliegue contenerizado con dos servicios:

Listing 3: Arranque del entorno con Docker Compose.

```
1 docker-compose up --build # levanta redis:7-alpine + app Flask en :5000
```

El servicio `web` depende de un *healthcheck* de Redis para garantizar que la base de datos esté lista antes de que la aplicación inicie.

13.2 Variables de Entorno

Tabla 13: Variables de entorno configurables.

Variable	Por defecto	Descripción
SECRET_KEY	dev-secret-key	Clave de sesión Flask
REDIS_HOST	localhost	Host de Redis
REDIS_PORT	6379	Puerto de Redis
REDIS_DB	0	Base de datos Redis
FLASK_ENV	development	Entorno de ejecución

13.3 Primer Uso

Al registrar el primer usuario, el sistema le asigna automáticamente el rol `superadmin`:

Listing 4: Asignación automática del rol al primer usuario.

```
1 es_primer = srp.num_objs(Usuario) == 0
2 rol = 'superadmin' if es_primer else 'normal'
```

14 Estructura del Proyecto

Listing 5: Árbol de directorios del proyecto.

```
1 src/
2 +-- app/
3 |   +-- __init__.py           # Application Factory
4 |   +-- decorators.py         # @superadmin_required,
   |   @acceso_vivienda_required
```

```

5 |   +-- helpers.py           # OIDs, cascadas, coeficientes,
   |   notificaciones
6 |   +-- models/             # 8 modelos Python (una clase por
   |   entidad)
7 |   +-- modules/            # 10 blueprints (un directorio por
   |   blueprint)
8 |   |   +-- auth/  main/  comunidades/  viviendas/
9 |   |   +-- usuarios/ accesos/  baterias/
10 |  |   +-- cierres/ incidencias/ notificaciones/
11 |  +-- static/
12 |  |   +-- css/custom.css   # Design system completo
13 |  |   +-- js/ajax.js      # Progressive enhancement AJAX
14 |  |   +-- js/charts.js    # Graficas Chart.js 4
15 |  +-- templates/          # 34 plantillas Jinja2
16 +-- tests/                 # 11 ficheros, 170 tests
17 +-- config.py              # Configuración por entorno
18 +-- run.py                 # Punto de entrada
19 +-- Dockerfile
20 +-- docker-compose.yml
21 +-- requirements.txt

```

15 Criterios del Enunciado — Tabla de Cumplimiento

Tabla 14: Cumplimiento de los criterios del enunciado de la asignatura.

Criterio	Implementación
Flask + Jinja2	Flask 3.0.3, 34 templates Jinja2
Sirope + Redis	Sirope 0.3.1, Redis 7
Flask-Login	Autenticación completa con sesión segura
Flask-WTF / CSRF	CSRF en todos los formularios, incluido AJAX
Diseño modular	10 blueprints, 7 modelos, 1 módulo por entidad
Pico CSS	Pico CSS 2 con design system custom
Sin recarga (AJAX)	<code>ajax.js</code> con progressive enhancement
Robustez	Páginas de error 401/403/404/500, validación WTForms
Integridad referencial	Cascadas implementadas, guardia de último titular
Navegable sin volver atrás	Breadcrumbs y enlaces en todas las vistas
Primer usuario es superadmin	<code>auth.registro</code> comprueba <code>num_objs(Usuario) == 0</code>
Tests	170 tests con pytest cubriendo los 10 módulos
Diagramas	Diagrama de clases y diagrama de casos de uso